Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №8

Дисциплина: Основы программирования на Python

Тема: Паттерны проектирования в Python

Вариант 2: Библиотека

Цель работы

Изучить назначение и получить практические навыки применения паттернов проектирования. В ходе выполнения работы необходимо научиться распознавать типовые проблемы проектирования, выбирать подходящий паттерн для решения конкретной проблемы, реализовывать паттерны на Python, комбинировать несколько паттернов в одном приложении, использовать абстракции и полиморфизм, применять композицию объектов, отделять создание объектов от их использования, заменять условную логику стратегиями, инкапсулировать операции в объекты-команды, использовать Repository как отдельный слой доступа к данным и оценивать преимущества и недостатки выбранного решения.

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать систему управления выдачей книг. С помощью порождающего паттерна Factory реализовать создание трёх типов позиций фонда: печатной книги, электронной книги и журнала. С помощью поведенческого паттерна Strategy реализовать три правила расчёта срока выдачи: стандартный срок, льготный срок и срок для преподавателей. Дополнительно применить паттерн Command для инкапсуляции операций выдачи, возврата и бронирования. Работа выполняется в три этапа: наивная реализация, анализ проблем расширения, выбор и реализация паттернов с последующим сравнением решений.

Краткие теоретические сведения

Паттерн проектирования — это описание типового способа решения часто встречающейся задачи проектирования. Паттерн не является готовым кодом: он задаёт структуру взаимодействия объектов и позволяет обсуждать архитектурные решения в единых терминах. По назначению паттерны делят на три группы.

- Порождающие паттерны управляют процессом создания объектов: Factory Method, Abstract Factory, Builder, Singleton, Prototype.
- Структурные паттерны описывают способы объединения классов и объектов: Adapter, Decorator, Facade, Composite, Proxy.
- Поведенческие паттерны распределяют обязанности между объектами: Strategy, Command, Observer, State, Template Method.

Factory выносит создание объектов в отдельный компонент, поэтому клиентский код работает с абстрактным типом и не зависит от конкретных классов. Strategy инкапсулирует семейство взаимозаменяемых алгоритмов в отдельных классах с общим интерфейсом, что позволяет заменять алгоритм во время выполнения без изменения контекста. Command превращает операцию в самостоятельный объект с методом execute(), благодаря чему операцию можно передавать, ставить в очередь и логировать. Repository отделяет бизнес-логику от деталей хранения данных.

Структура проекта

Компонент	Назначение
models/enums.py	Перечисления ItemType, ItemStatus, ReaderCategory
models/entities.py	dataclass LibraryItem и производные PrintedBook, EBook, Magazine; Reader и LoanRecord
interfaces/	Протоколы Repository, ItemFactory, LoanStrategy
repositories/memory_repository.py	Хранилище позиций фонда в памяти (паттерн Repository)
factories/library_factory.py	Фабрика LibraryItemFactory с реестром классов (паттерн Factory)
strategies/loan_strategies.py	Три стратегии расчёта срока выдачи и функция выбора стратегии

Компонент	Назначение
commands/library_commands.py	Абстрактный класс Command и команды IssueCommand, ReturnCommand, ReserveCommand
services/library_service.py	Бизнес-логика: добавление, поиск, фильтрация, выдача, возврат, бронирование, статистика
ui/console.py, main.py	Консольное меню и точка сборки приложения с демонстрационными данными

Анализ первоначальной реализации

Наивная реализация предметной области строится на условных конструкциях. Создание позиции фонда выполняется цепочкой if/elif по строковому типу: если тип равен «printed_book», создаётся печатная книга, если «ebook» — электронная, если «magazine» — журнал. Расчёт срока выдачи также выполняется ветвлением по категории читателя, а операции выдачи, возврата и бронирования реализуются отдельными методами сервиса с повторяющимися проверками состояния.

При расширении программы такой код создаёт три проблемы. Во-первых, добавление нового типа позиции (например, аудиокниги) требует правки сразу нескольких условных конструкций в разных методах, что нарушает принцип открытости/закрытости. Во-вторых, правила расчёта срока выдачи оказываются «зашиты» в метод выдачи, поэтому их нельзя переиспользовать и трудно тестировать отдельно. В-третьих, операции предметной области не существуют как самостоятельные объекты: их нельзя журналировать, отменять или вызывать из другого интерфейса без дублирования кода.

Выбор паттернов

Для каждой проблемы выбран паттерн, устраняющий её причину, а не следствие.

Проблема	Паттерн	Причина выбора и структура решения
Создание позиций разных типов через if/elif	Factory	Клиент передаёт ItemType, фабрика по реестру выбирает класс и создаёт объект; новый тип добавляется одной записью в реестре

Проблема	Паттерн	Причина выбора и структура решения
Ветвление по категории читателя при расчёте срока	Strategy	Каждый срок выдачи оформлен отдельным классом с методом calculate_days(); сервис получает стратегию и не знает её конкретный тип
Операции выдачи, возврата и бронирования дублируются и не логируются	Command	Каждая операция инкапсулирована в класс с методом execute(); появляется единая точка проверки состояния и журналирования
Бизнес-логика напрямую работает с коллекцией объектов	Repository	Доступ к данным вынесен за протокол Repository с методами add, get, get_all, update, delete

Реализация паттерна Factory

Фабрика LibraryItemFactory хранит словарь-реестр, сопоставляющий значение перечисления ItemType конкретному классу. Статический метод create() находит класс по ключу и вызывает его конструктор с переданными именованными аргументами; при неизвестном типе возбуждается ValueError. Клиентский код сервиса вызывает LibraryItemFactory.create(item_type, **kwargs) и получает объект базового типа LibraryItem, не завися от конкретных классов позиций.

Листинг 8.1 — Фабрика с реестром классов (factories/library_factory.py)

```
class LibraryItemFactory:
    _registry = {
        ItemType.PRINTED_BOOK: PrintedBook,
        ItemType.EBOOK: EBook,
        ItemType.MAGAZINE: Magazine,
    }

    @staticmethod
    def create(item_type: ItemType, **kwargs) -> LibraryItem:
        cls = LibraryItemFactory._registry.get(item_type)
        if cls is None:
            raise ValueError(f"Unknown item type: {item_type}")
        return cls(**kwargs)
```

Реализация паттерна Strategy

Срок выдачи рассчитывается тремя стратегиями: StandardLoanStrategy возвращает 14 дней, PreferentialLoanStrategy — 30 дней для льготной категории, TeacherLoanStrategy — 60 дней для преподавателей. Функция

strategy_for_reader() выбирает подходящую реализацию по категории читателя, а команда выдачи вызывает у полученного объекта метод calculate_days(). Добавление нового правила, например срока для студентов заочного отделения, сводится к созданию нового класса стратегии и одной строке выбора.

Листинг 8.2 — Стратегии расчёта срока выдачи (strategies/loan_strategies.py)

```
class StandardLoanStrategy:
    name = "standard"

    def calculate_days(self, reader: Reader) -> int:
        return 14

class PreferentialLoanStrategy:
    name = "preferential"

    def calculate_days(self, reader: Reader) -> int:
        if reader.category == ReaderCategory.PREFERENTIAL:
            return 30
        return 14

class TeacherLoanStrategy:
    name = "teacher"

    def calculate_days(self, reader: Reader) -> int:
        if reader.category == ReaderCategory.TEACHER:
            return 60
        return 14

def strategy_for_reader(reader: Reader):
    if reader.category == ReaderCategory.TEACHER:
        return TeacherLoanStrategy()
    if reader.category == ReaderCategory.PREFERENTIAL:
        return PreferentialLoanStrategy()
    return StandardLoanStrategy()
```

Реализация паттерна Command

Абстрактный класс Command объявляет метод execute() и атрибут name. Классы IssueCommand, ReturnCommand и ReserveCommand получают в конструкторе ссылку на репозиторий и параметры операции, а в методе execute() выполняют проверку состояния позиции, изменяют статус и возвращают текстовый отчёт. Сервис LibraryService не содержит предметной логики этих операций: он лишь создаёт соответствующую команду и вызывает её метод execute(). Благодаря этому проверки состояния сосредоточены в одном месте, а саму операцию можно передать другому компоненту интерфейса.

Листинг 8.3 — Команда выдачи (commands/library_commands.py)

```
class Command(ABC):
    name: str = "command"

    @abstractmethod
    def execute(self) -> Optional[str]:
        ...

class IssueCommand(Command):
    name = "issue"

    def __init__(self, repo, item_id: int, reader: Reader):
        self.repo = repo
        self.item_id = item_id
        self.reader = reader

    def execute(self) -> str:
        item = self.repo.get(self.item_id)
        if item is None:
            raise ValueError(f"Item {self.item_id} not found")
        if item.status != ItemStatus.AVAILABLE:
            raise ValueError(f"Item {self.item_id} is not available")

        strategy = strategy_for_reader(self.reader)
        days = strategy.calculate_days(self.reader)

        item.status = ItemStatus.ISSUED
        self.repo.update(item)

        record = LoanRecord(item_id=item.id, reader_id=self.reader.id,
                            issued_at=date.today(),
                            due_at=date.today() + timedelta(days=days))
        return (f"Выдана позиция «{item.title}» читателю "
                f"{self.reader.name}. Срок: {days} дн. "
                f"(до {record.due_at})")
```

Сравнение решений

Критерий	Наивная реализация	Реализация с паттернами
Добавление нового типа позиции	Правка условных конструкций в нескольких методах	Одна запись в реестре фабрики
Добавление правила срока выдачи	Изменение метода выдачи	Новый класс стратегии и одна строка выбора
Журналирование операций	Дублирование кода в каждом методе	Единая точка в методе execute() команды
Тестируемость	Требуется полностью собранный сервис	Фабрика, стратегии и команды тестируются изолированно
Замена хранилища	Правка сервиса	Новая реализация протокола Repository

Раздел тестирования

Для проверки реализации написано одиннадцать автоматических тестов на основе pytest с фикстурой service, собирающей полный стек приложения: хранилище в памяти, сервис и демонстрационные данные. Тесты проверяют как работу отдельных паттернов, так и предметные сценарии.

№	Тест	Проверяемое поведение
1	test_factory_creates_printed_book	Фабрика по ItemType.PRINTED_BOOK создаёт объект PrintedBook
2	test_repository_add_and_get	Позиция сохраняется в хранилище и извлекается по идентификатору
3	test_repository_rejects_duplicate	Повторное добавление позиции с тем же ID отклоняется
4	test_strategy_for_teacher_gives_60_days	Для преподавателя стратегия возвращает 60 дней
5	test_strategy_for_preferential_gives_30_days	Для льготной категории — 30 дней
6	test_strategy_for_student_gives_14_days	Для студента — стандартные 14 дней
7	test_issue_changes_status	Команда выдачи переводит позицию в статус ISSUED
8	test_cannot_issue_reserved_item	Выдача забронированной позиции приводит к ошибке
9	test_return_restores_available	Команда возврата восстанавливает статус AVAILABLE
10	test_find_by_author	Поиск по автору возвращает корректный список позиций
11	test_statistics	Статистика верно подсчитывает общее число, доступные, выданные и забронированные позиции

Все тесты проходят успешно. Дополнительно работа проверялась вручную через консольное меню: добавление печатной книги, электронной книги и журнала, выдача позиции читателям разных категорий с проверкой рассчитанного срока, бронирование, возврат, поиск по автору и названию, фильтрация по статусу и вывод статистики.

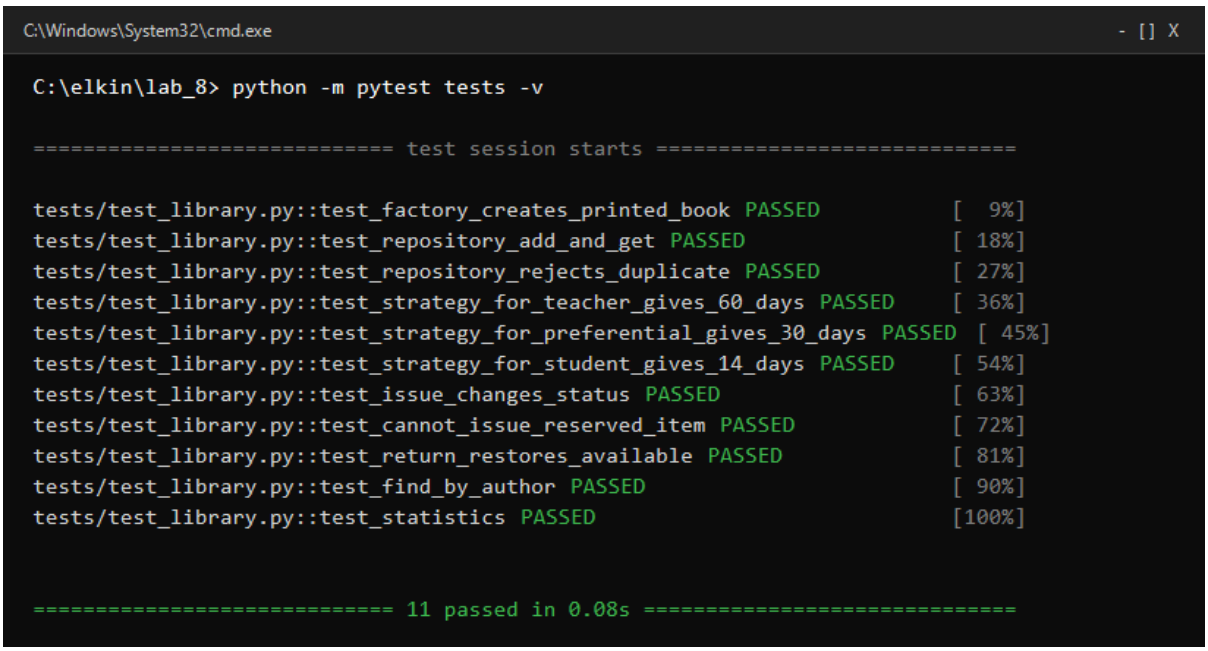


Рисунок 8.1 — Результат прохождения тестов фабрики, стратегий и команд

Листинги ключевых фрагментов программы

Листинг 8.4 — Сервис, использующий фабрику и команды (services/library_service.py)

```
class LibraryService:
    def __init__(self, repository: Repository):
        self.repository = repository


    def add_item(self, item_type: ItemType, **kwargs) -> LibraryItem:
        item = LibraryItemFactory.create(item_type, **kwargs)
        self.repository.add(item)
        return item


    def issue(self, item_id: int, reader: Reader) -> str:
        return IssueCommand(self.repository, item_id, reader).execute()


    def return_item(self, item_id: int) -> str:
        return ReturnCommand(self.repository, item_id).execute()


    def reserve(self, item_id: int) -> str:
        return ReserveCommand(self.repository, item_id).execute()
```

Листинг 8.5 — Протоколы слоя абстракций (interfaces/)

```
class Repository(Protocol):
    def add(self, item: LibraryItem) -> None: ...
    def get(self, item_id: int) -> Optional[LibraryItem]: ...
    def get_all(self) -> list[LibraryItem]: ...
    def delete(self, item_id: int) -> bool: ...
    def update(self, item: LibraryItem) -> None: ...


class LoanStrategy(Protocol):
    def calculate_days(self, reader: Reader) -> int: ...
```

Вывод

В ходе выполнения лабораторной работы изучены назначение и практика применения паттернов проектирования. На примере предметной области «Библиотека» реализованы порождающий паттерн Factory, поведенческие паттерны Strategy и Command, а также слой доступа к данным на основе Repository. Сравнение наивной реализации с итоговой показало, что паттерны устраняют ветвления по типу объекта, локализуют правила предметной области в отдельных классах и делают добавление новых возможностей расширением, а не изменением существующего кода. Комбинирование нескольких паттернов в одном приложении позволило получить гибкую архитектуру, готовую к автоматическому тестированию.